
Oliver Krauss

Aggregation II

MDB3 | MTD.ba VZ WS26

Hagenberg 13.10.2026

HAGENBERG | LINZ | STEYR | WELS



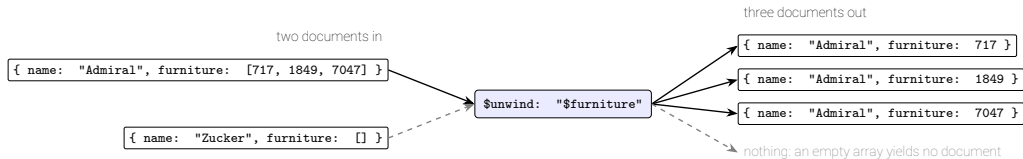
UNIVERSITY
OF APPLIED SCIENCES
UPPER AUSTRIA

The Question a Junction Table Would Answer

```
SELECT item_id, COUNT(*) AS owners FROM villager_furniture  
GROUP BY item_id ORDER BY owners DESC LIMIT 5;
```

- "Which items do most villagers own?" In SQL that is one **GROUP BY** over the junction table
- In the catalogue there is no junction table: each villager carries a **furniture** array of item ids
- **\$group** on "**\$furniture**" would group by the **whole array**. We need one document *per element* first

\$unwind, as a Picture



- One document with an array of three becomes **three documents**, each a full copy with one element in place of the array
- Every other field is copied along: **name** appears three times, and so would **species**, **home**, all of it
- An empty or missing array yields nothing. Zucker is gone after this stage, unless you ask to keep him

\$unwind: One Document per Array Element

```
db.villagers.aggregate([ { $count: "villagers" } ]) // 391
db.villagers.aggregate([ { $unwind: "$furniture" }, { $count: "rows" } ]) // 4804

db.villagers.aggregate([ { $match: { name: "Admiral" } }, { $unwind: "$furniture" },
                        { $project: { _id: 0, name: 1, furniture: 1 } } ])
// { name: 'Admiral', furniture: 717 }
// { name: 'Admiral', furniture: 1849 }
// { name: 'Admiral', furniture: 7047 } ... twelve in all
```

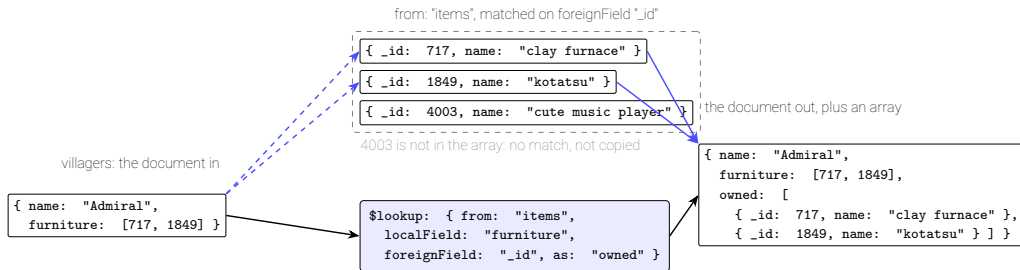
- **\$unwind** copies the document once per element and replaces the array with that element
- 391 villagers become 4,804 documents: **the junction table, computed on the fly**
- A document with an empty or missing array disappears (**preserveNullAndEmptyArrays** keeps it)

Unwind, then Group

```
db.villagers.aggregate([
  { $unwind: "$furniture" },
  { $group: { _id: "$furniture", owners: { $sum: 1 } } },
  { $sort: { owners: -1, _id: 1 } },
  { $limit: 5 }
])
// { _id: 4003, owners: 79 } { _id: 2554, owners: 48 } { _id: 3672, owners: 47 }
// { _id: 4338, owners: 45 } { _id: 794, owners: 41 }
```

- After the **\$unwind** the pipeline is the **GROUP BY** from the first slide, stage for stage
- **\$unwind** before **\$group** is the pattern for every “per element” question: per tag, per colour, per material
- The answer is item ids. The names live in another collection

\$lookup, as a Picture



- For each document in, the stage looks up every **items** document whose **_id** equals a value of **furniture**
- The matches are **added as an array** under the name in **as**; the document itself is unchanged
- No match: an empty array, and the document still comes out. That is the **LEFT OUTER** in the join

\$lookup: LEFT OUTER JOIN, One Stage

```
SELECT v.name, i.name FROM villagers v LEFT JOIN items i ON i.id = ANY(v.furniture)
```

```
db.villagers.aggregate([
  { $match: { name: "Admiral" } },
  { $lookup: { from: "items", localField: "furniture", foreignField: "_id", as: "
    owned" } },
  { $project: { _id: 0, name: 1, n: { $size: "$owned" }, first: { $first: "$owned.
    name" } } }
])
// { name: 'Admiral', n: 12, first: 'kotatsu' }
```

- from the other collection, **localField** = **foreignField**, result as a new **array field**
- Left outer: a villager without matches keeps an empty **owned**. A local array matches **any element**: twelve ids, twelve items

The Full Report: Unwind, Group, Lookup, Shape

```
db.villagers.aggregate([
  { $unwind: "$furniture" },
  { $group: { _id: "$furniture", owners: { $sum: 1 } } },
  { $sort: { owners: -1, _id: 1 } }, { $limit: 5 },
  { $lookup: { from: "items", localField: "_id", foreignField: "_id", as: "item" } },

  { $unwind: "$item" },
  { $project: { _id: 0, item: "$item.name", sell: "$item.sell", owners: 1 } }
])
// { owners: 79, item: 'cute music player', sell: 650 }
// { owners: 48, item: 'double sofa', sell: 1075 }
// { owners: 47, item: 'cardboard box', sell: 30 } ...
```

- **\$lookup after** the **\$limit**: five joins instead of 4,804. Join late, join little
- The join result is an array of one; the second **\$unwind** makes it a plain embedded document

The Join the Other Way, and When Not to Join

```
db.items.aggregate([
  { $match: { _id: { $in: [717, 1849] } } },
  { $lookup: { from: "villagers", localField: "_id", foreignField: "furniture", as:
    "owners" } },
  { $project: { _id: 0, name: 1, owners: "$owners.name" } }
])
// { name: 'clay furnace', owners: [ 'Admiral', 'Bill', 'Dora', 'Gaston', ... ] }
// { name: 'kotatsu', owners: [ 'Admiral', 'Humphrey', 'Marcel', 'Wart Jr.' ] }
```

- **foreignField** can be an array too: “every villager whose **furniture** contains this **_id**”
- “**\$owners.name**” on an array of documents yields the array of names: no unwind needed for a list
- **When not to join:** the island page needs resident names on every load. That was the extended reference in *Document Modelling*: copy the name, skip the **\$lookup**. Joins are for reports, not for pages

Translate This SQL

With your neighbour. Write the pipeline. The junction table `villager_furniture(villager_id, item_id)` is the `furniture` array in the catalogue.

1. `SELECT species, COUNT(*) FROM villagers v JOIN villager_furniture f ON f.villager_id = v.id GROUP BY species ORDER BY 2 DESC LIMIT 3`
2. `SELECT i.name FROM items i JOIN villager_furniture f ON f.item_id = i.id WHERE i.type = 'houseware' GROUP BY i.id, i.name HAVING COUNT(*) >= 40`
3. Every villager who owns nothing that is also owned by Admiral. (Take your time. What is the first stage?)

Translate This SQL: Answers

```
// 1: unwind, then the usual group. Rabbit 261, Cat 258, Squirrel 247 ownerships
[ { $unwind: "$furniture" }, { $group: { _id: "$species", n: { $sum: 1 } } },
  { $sort: { n: -1 } }, { $limit: 3 } ]
```

```
// 2: count, join only the survivors, filter by type. Only the double sofa (48)
[ { $unwind: "$furniture" }, { $group: { _id: "$furniture", n: { $sum: 1 } } },
  { $match: { n: { $gte: 40 } } },
  { $lookup: { from: "items", localField: "_id", foreignField: "_id", as: "i" } },
  { $unwind: "$i" }, { $match: { "i.type": "houseware" } },
  { $project: { _id: 0, name: "$i.name", n: 1 } } ]
```

```
// 3: no pipeline needed. Two queries: Admiral's array, then $nin on it.
const a = db.villagers.findOne({ name: "Admiral" }).furniture
db.villagers.find({ furniture: { $not: { $elemMatch: { $in: a } } } }, { name: 1 })
```

- Number 3 is the lesson: a pipeline is not the only tool. When one document holds the key, read it and query

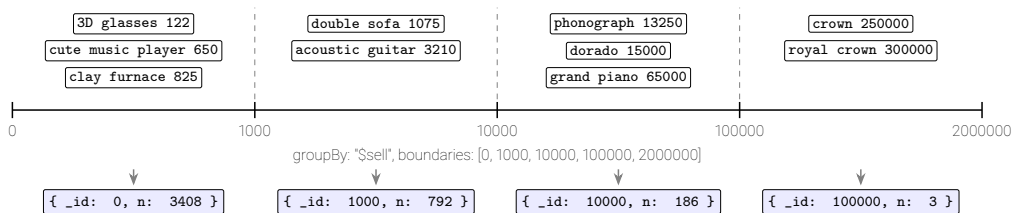
Group by an Expression

```
db.villagers.aggregate([
  { $group: { _id: "$birthday.month", n: { $sum: 1 } } },
  { $sort: { n: -1 } }, { $limit: 3 }
])
// { _id: 10, n: 37 } { _id: 8, n: 36 } { _id: 7, n: 35 }

{ $group: { _id: { $substrCP: ["$name", 0, 1] }, n: { $sum: 1 } } } // letter
{ $group: { _id: { $cond: [ { $gte: ["$sell", 10000] }, "dear", "cheap" ] },
  n: { $sum: 1 } } } // classes
```

- The group key is any **expression**: a dot path, a string function, a condition, a date part (\$month)
- SQL: GROUP BY MONTH(birthday), GROUP BY LEFT(name, 1), GROUP BY CASE WHEN ...
- Classes by hand work for two or three; for ranges there is a stage

\$bucket, as a Picture



- The boundaries cut the number line into bins; a document falls into the bin its **sell** lies in, lower edge included
- Out comes **one document per bin**, named after its lower edge, with what the **output** accumulators computed

\$bucket: GROUP BY Range

```
db.items.aggregate([
  { $match: { sell: { $exists: true } } },
  { $bucket: { groupBy: "$sell", boundaries: [0, 1000, 10000, 100000, 2000000],
    default: "other",
    output: { n: { $sum: 1 }, examples: { $firstN: { n: 3, input: "$name" } } } } }
])
// { _id: 0,      n: 3408, examples: [ '3D glasses', 'bandage', 'beak' ] }
// { _id: 1000,   n: 792,  examples: [ 'DAL eye mask', ... ] }
// { _id: 10000, n: 186 } { _id: 100000, n: 3, examples: [ 'crown', ... ] }
```

- **boundaries**: each bucket is [a, b). Values outside go to **default**, or the stage errors
- **output** takes the accumulators of **\$group**. **\$bucketAuto** picks the boundaries itself

\$facet: Several Reports in One Pass

```
db.items.aggregate([
  { $match: { type: "fish" } },
  { $facet: {
    byHabitat: [ { $sortByCount: "$whereHow" }, { $limit: 3 } ],
    priciest:  [ { $sort: { sell: -1 } }, { $limit: 3 },
                  { $project: { _id: 0, name: 1, sell: 1 } } ],
    total:    [ { $count: "fish" } ]
  } }
])
// { byHabitat: [ { _id: 'Sea', count: 29 }, { _id: 'River', count: 27 }, ... ],
//   priciest: [ { name: 'dorado', sell: 15000 }, ... ],   total: [ { fish: 80 } ] }
```

- Each facet is a **sub-pipeline** on the same input; the result is one document with one array per facet
- A page with “by habitat”, “top prices” and the total: one round trip instead of three.
\$sortByCount is group, count and sort in one stage

Predict the Output

Your turn. Shape and count of the result, then the first document.

```
// 1
db.villagers.aggregate([ { $match: { species: "Octopus" } },
  { $unwind: "$colors" },
  { $group: { _id: "$colors", who: { $addToSet: "$name" } } }
])

// 2
db.villagers.aggregate([
  { $match: { name: "Admiral" } },
  { $lookup: { from: "items", localField: "furniture", foreignField: "_id",
    as: "owned" } },
  { $unwind: "$owned" },
  { $group: { _id: "$name", value: { $sum: "$owned.sell" }, n: { $sum: 1 } } }
])

// 3: what changes if the $unwind in 2 is removed?
```

Predict the Output: Answers

```
// 1: six documents, one per colour of the three octopuses
{ _id: 'White', who: [ 'Octavian' ] } { _id: 'Red', who: [ 'Marina' ] } ...

// 2: one document: Admiral's twelve items, summed. The $lookup array was unwound
    first.
{ _id: 'Admiral', value: 10542, n: 12 }

// 3: without the $unwind, "$owned.sell" is an array of twelve numbers.
//     $sum of an array in $group: not a number -> 0. n: 1. No error.
{ _id: 'Admiral', value: 0, n: 1 }
```

-- Number 3 again: a wrong shape is not an error. Look at the stage before the surprise

\$merge and \$out: Materialised Reports

```
db.villagers.aggregate([
  { $unwind: "$furniture" },
  { $group: { _id: "$furniture", owners: { $sum: 1 } } },
  { $merge: { into: "itemPopularity", whenMatched: "replace", whenNotMatched: "
    insert" } }
])
db.itemPopularity.find().sort({ owners: -1 }).limit(3)  // 755 documents, queryable,
indexable
```

- **\$out** replaces a collection with the result; **\$merge** updates it document by document and can run again tonight
- The report becomes a collection: a page reads it with **find()** in a millisecond instead of unwinding 4,804 rows per request
- SQL calls this a materialised view. Refresh it on a schedule, and know that it is as old as its last run

Reading a Pipeline You Did Not Write

- **Find the shape changes.** `$unwind` multiplies, `$group` collapses, `$lookup` adds an array, `$facet` nests. Everything else keeps one document per document
- **Check what each stage sees.** After a `$group` only `_id` and the accumulators exist; after `$unwind` the array is a value; after `$lookup` the join is an array until you unwind it
- **Join late.** `$match` and `$limit` before `$lookup`; nobody joins 4,804 rows to read five
- **Ask what SQL it is.** `$unwind` + `$group` is a junction-table `GROUP BY`; `$bucket` is `GROUP BY CASE`; `$facet` is three queries in one

Quick Retrieval: Test Yourself

Before the summary: can you answer these from memory?

1. 391 villagers, 4,804 ownerships. Which stage produces the second number from the first?
2. What does **\$lookup** put into the document, and what do you usually do with it next?
3. Why does **\$lookup** come after **\$limit** in the top-five report?
4. When is a **\$lookup** the wrong tool even though the data is in two collections?
5. **\$bucket** boundaries **[0, 1000, 10000]**: which bucket is 1000 in? What happens to 50000?
6. **\$out** versus **\$merge**: which one can you run every night without losing anything?

Think first, then we go through the answers.

Quick Retrieval: Answers

1. **\$unwind** on **furniture**: one document per element.
2. An array of matching documents from the other collection; usually **\$unwind** it, or read a field with "**\$arr.field**".
3. Five joins instead of 4,804. Join late, join little.
4. When a page needs the joined field on every load: copy it (extended reference) instead.
5. In **[1000, 10000)**: boundaries are inclusive at the bottom. 50000 goes to **default**, and without a **default** the stage fails.
6. **\$merge**: it updates or inserts per document. **\$out** replaces the whole collection.

Summary: Aggregation II

Stages

- **\$unwind**: the junction table, computed
- **\$lookup**: a left outer join into an array; unwind it or read "**\$arr.field**"
- **\$bucket**, **\$bucketAuto**: **GROUP BY** range
- **\$facet**: several sub-pipelines, one result
- **\$merge**, **\$out**: the report as a collection

Judgement

- Group keys are expressions: month, first letter, a condition
- Join late and little; for pages, copy instead of joining
- A wrong shape is never an error. Read stage by stage
- Next: making it fast, and making it refuse bad data (*Indexes and Schema Validation*)

Details: \$unwind Options

```
{ $unwind: { path: "$furniture", includeArrayIndex: "pos",  
            preserveNullAndEmptyArrays: true } }  
// { name: 'Admiral', furniture: 717, pos: 0 } ...  
// a villager with furniture: [] or no field at all: { name: ..., pos: null }
```

- **includeArrayIndex** keeps the position, for “the first three items” or ordered lists
- **preserveNullAndEmptyArrays** keeps documents that would otherwise vanish: the left outer join of unwinding
- Unwinding a non-array value emits the document unchanged, once

Details: \$lookup with a Pipeline

```
{ $lookup: { from: "items", let: { ids: "$furniture" },  
  pipeline: [ { $match: { $expr: { $in: ["$_id", "$$ids"] } } },  
    { $match: { sell: { $gte: 10000 } } },  
    { $project: { name: 1, sell: 1 } } ],  
  as: "expensiveOwned" } }
```

- **let** passes fields of the outer document as variables (**\$\$ids**); the inner **pipeline** filters, projects, even groups
- The join condition lives in **\$expr**; anything else is a plain stage on the foreign collection
- Since 5.0 the two forms combine: **localField/foreignField** plus a **pipeline** for the rest
- Also in the family: **\$graphLookup** (recursive, “friends of friends”) and **\$unionWith** (**UNION ALL**)

Details: \$bucketAuto and \$setWindowFields

```
{ $bucketAuto: { groupBy: "$sell", buckets: 4, output: { n: { $sum: 1 } } } }  
// { _id: { min: 0, max: 222 }, n: 1104 } { _id: { min: 222, max: 380 }, n: 1172 }  
...  
  
{ $setWindowFields: { partitionBy: "$type", sortBy: { sell: -1 },  
  output: { rank: { $rank: {} }, runningTotal: { $sum: "$sell",  
    window: { documents: ["unbounded", "current"] } } } } }
```

- **\$bucketAuto** chooses boundaries so buckets are about equal in count; **granularity** snaps them to round numbers
- **\$setWindowFields** (5.0+) are SQL's window functions: rank, running totals, moving averages, per partition

Details: Views and On-Demand Materialised Views

```
db.createView("fishByHabitat", "items", [
  { $match: { type: "fish" } }, { $sortByCount: "$whereHow" } ])
db.fishByHabitat.find()           // the pipeline runs on every read

{ $merge: { into: "itemPopularity", on: "_id",
  whenMatched: "replace", whenNotMatched: "insert" } }
```

- A view is a saved pipeline with a name: always fresh, never faster than the pipeline
- **\$merge** is the on-demand materialised view: as fast as a collection, as fresh as its last run
- **\$merge** can target another database, and **whenMatched** can be a pipeline (add counts instead of replacing)